

Lab Assignment 5 by Dalton JS Crawford

This lab explored the parallelization of a prime-counting program using MPI with a Master/Worker architecture to address load balancing challenges inherent in the problem. The core difficulty arises from the fact that checking primality is not uniform across numbers. Larger numbers require more iterations in the trial division loop, meaning naïvely splitting the range evenly across ranks results in significant load imbalance. To solve this, the Master dynamically dispatches small chunks of work to workers as they become available, ensuring that no rank sits idle while others are overloaded. The chunk size was treated as a tunable parameter, and its effect on performance was measured across a wide range of values from 10 to 1,000,000.

The results demonstrate the practical tradeoffs between chunk size, load balance, and communication overhead. As shown in the first plot, very small chunk sizes increase MPI communication frequency and introduce overhead that degrades performance, while very large chunks reintroduce load imbalance, though this was not as noticeable given that if the chunk is larger than the sample, parallelism is avoided entirely. The optimal chunk size sits in a middle range that minimizes total runtime. The strong scaling plots further illustrate this contrast. The load imbalanced version shows poor scaling as rank count increases due to uneven work distribution, while the dynamically load balanced version using the optimal chunk size achieves notably better scaling, with runtime decreasing more consistently as additional ranks are added.





